

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



BRILYANT KEYZA HIDAYAT

109082500106

S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori (Selection Sort)

Selection Sort adalah metode pengurutan dengan prinsip utama mencari nilai ekstrim (terkecil atau terbesar) pada sekumpulan data tersisa, lalu menempatkannya di posisi indeks yang seharusnya. Algoritma ini bekerja dengan membagi array secara visual menjadi dua bagian: bagian yang sudah terurut dan bagian yang belum terurut.

Karakteristik utama dari Selection Sort meliputi beberapa proses berikut:

- Memindai rentang data tersisa untuk mencari nilai terkecil untuk urutan membesar (*ascending*) atau terbesar untuk urutan mengecil (*descending*).
- Memindahkan atau menukar posisi indeks nilai ekstrim yang ditemukan dengan elemen yang berada pada posisi paling kiri pada rentang data tersisa tersebut.
- Proses pencarian dan pertukaran ini diulang secara terus-menerus hingga rentang data yang belum terurut hanya tersisa satu data saja.

B. Dasar Teori (Insertion Sort)

Insertion Sort adalah metode pengurutan dengan prinsip utama menyisipkan suatu nilai pada posisi yang tepat. Berbeda dengan Selection Sort, algoritma ini tidak mencari nilai ekstrim terlebih dahulu, melainkan mengambil elemen secara berurutan lalu mencari posisi selipnya menggunakan pencarian sekuensial (*Sequential Search*).

Karakteristik utama dari Insertion Sort meliputi beberapa proses berikut:

- Menyimpan nilai dari elemen yang sedang dievaluasi ke dalam variabel sementara agar nilainya tidak hilang saat proses pemindahan.
- Menggeser data yang sudah terurut ke arah kanan untuk menciptakan ruang kosong bagi elemen belum terurut yang akan dimasukkan.
- Memasukkan nilai dari variabel temporer ke dalam ruang kosong yang telah disiapkan di antara rangkaian data yang sudah terurut dengan tetap menjaga keterurutan data.

C. Guided

1. SelectionSortingArray

```
package main
import "fmt"

func SelectionSortArray(arr *[8]int){
    var idx_min, i, j int
    for i = 0; i < len(arr); i++ { //perulangan luar
        idx_min = i;
        for j = i + 1; j < len(arr); j++ { //perulangan
dalam
            if arr[j] < arr[idx_min] { //kondisional
                idx_min = j
            }
        }
        if idx_min != i { //swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main(){
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka index ke-%d :
", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Guid
ed\Guided - 1\soal1.go"
Masukkan data angka index ke-0 : 3
Masukkan data angka index ke-1 : 7
Masukkan data angka index ke-2 : 4
Masukkan data angka index ke-3 : 1
Masukkan data angka index ke-4 : 8
Masukkan data angka index ke-5 : 9
Masukkan data angka index ke-6 : 4
Masukkan data angka index ke-7 : 2

=== SEBELUM SORTING ===
3 7 4 1 8 9 4 2
=== SETELAH SORTING ===
1 2 3 4 4 7 8 9
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat>
```

2. SelectionSortingStruct

```
package main
import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK float64
}

func SelectionSortArray(sMHS *[5]mahasiswa){
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //perulangan luar
        idx_min = i;
        for j = i + 1; j < len(sMHS); j++ { //loop
            dalam mencari nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                //sorting terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min],
sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d -
--\n", i)
        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)
```

```

        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Alpro Vscode\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscode\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Guided\Guided - 2\soal2.go"
--- Masukkan Data Mahasiswa Ke-0 ---
Nama: k
NIM: 123
IPK: 3.40
--- Masukkan Data Mahasiswa Ke-1 ---
Nama: e
NIM: 345
IPK: 4.0
--- Masukkan Data Mahasiswa Ke-2 ---
Nama: y
NIM: 567
IPK: 2.80
--- Masukkan Data Mahasiswa Ke-3 ---
Nama: z
NIM: 789
IPK: 3.74
--- Masukkan Data Mahasiswa Ke-4 ---
Nama: a
NIM: 1123
IPK: 2.20

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama: k
NIM: 123
IPK: 3.40
--- Data Mahasiswa Ke-1 ---
Nama: e
NIM: 345
IPK: 4.00
--- Data Mahasiswa Ke-2 ---
Nama: y
NIM: 567
IPK: 2.80
--- Data Mahasiswa Ke-3 ---
Nama: z
NIM: 789
IPK: 3.74
--- Data Mahasiswa Ke-4 ---
Nama: a
NIM: 1123
IPK: 2.20
=====
```

```
=== SETELAH SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama: a
NIM: 1123
IPK: 2.20
--- Data Mahasiswa Ke-1 ---
Nama: y
NIM: 567
IPK: 2.80
--- Data Mahasiswa Ke-2 ---
Nama: k
NIM: 123
IPK: 3.40
--- Data Mahasiswa Ke-3 ---
Nama: z
NIM: 789
IPK: 3.74
--- Data Mahasiswa Ke-4 ---
Nama: e
NIM: 345
IPK: 4.00
=====
```

3. InsertionSortingDescending

```
package main
import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
```

```

var n, i int

fmt.Print("Masukkan jumlah data: ")
fmt.Scan(&n)

fmt.Println("Masukkan data:")

for i = 0; i < n; i++ {
    fmt.Scan(&data[i])
}

fmt.Println("\nData sebelum sorting:")
for i = 0; i < n; i++ {
    fmt.Print(data[i], " ")
}

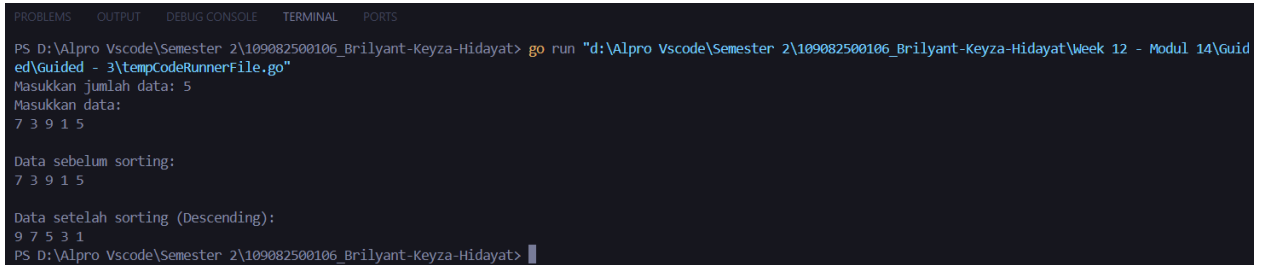
insertionSort(&data, n)

fmt.Println("\n\nData setelah sorting
(Descending):")
for i = 0; i < n; i++ {
    fmt.Print(data[i], " ")
}

fmt.Println()
}

```

Output :



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Guid
ed\Guided - 3\tempCodeRunnerFile.go"
Masukkan jumlah data: 5
Masukkan data:
7 3 9 1 5

Data sebelum sorting:
7 3 9 1 5

Data setelah sorting (Descending):
9 7 5 3 1
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat>

```

4. InsertionSortingStruct

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

```

```

}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM   : ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK   : ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Println("\nData sebelum sorting:")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim,
data[i].ipk)
    }

    insertionSortStruct(&data, n)
}

```



```
        fmt.Println("\nData setelah sorting (Descending
berdasarkan Nama):")
        for i = 0; i < n; i++ {
            fmt.Println(data[i].nama, data[i].nim,
data[i].ipk)
        }
    }
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Alpro Vscore\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscore\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Guided\Guided - 4\tempCodeRunnerFile.go"
Masukkan jumlah mahasiswa: 3

Data mahasiswa ke- 1
Nama : a
NIM  : 123
IPK  : 4

Data mahasiswa ke- 2
Nama : b
NIM  : 123
IPK  : 4

Data mahasiswa ke- 3
Nama : c
NIM  : 123
IPK  : 4

Data sebelum sorting:
a 123 4
b 123 4
c 123 4

Data setelah sorting (Descending berdasarkan Nama):
c 123 4
b 123 4
a 123 4
PS D:\Alpro Vscore\Semester 2\109082500106_Brilyant-Keyza-Hidayat> █
```

D. Unguided

1. Soal Unguided 1

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int) {
    var idx_min, i, j int
    for i = 0; i < len(arr); i++ {
        idx_min = i;
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        arr[i], arr[idx_min] = arr[idx_min], arr[i]
    }
}
```

```

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

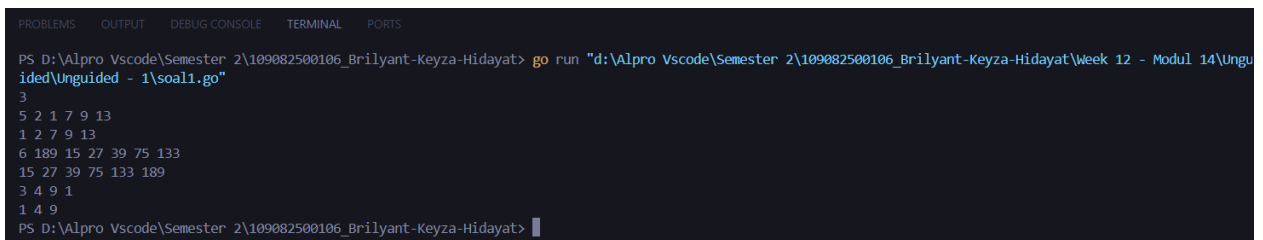
        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(rumah)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[j])
        }
        fmt.Println()
    }
}

```

Output :



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Ungu
ided\Unguided - 1\soal1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat>

```

Penjelasan :

Program ini dibuat untuk mengurutkan data nomor rumah di beberapa daerah menggunakan metode **Selection Sort** secara meningkat (*ascending*). Pertama program akan meminta input nilai *n* yang menentukan seberapa banyak daerah yang akan diproses. Untuk setiap daerahnya, program membaca jumlah rumah yang ada (*m*), lalu menyiapkan sebuah wadah dinamis (*slice*) berukuran *m* untuk menampung semua nomor rumah tersebut. Pada fungsi `selectionSort`, program akan mengecek nomor rumah satu per satu untuk mencari mana nomor yang paling kecil di dalam rentang data yang tersisa. Begitu nomor terkecil ditemukan,

setelah itu posisinya langsung ditukar ke sisi paling kiri. Proses ini terus diulangi sampai seluruh data terurut rapi dari terkecil ke terbesar. Setelah itu terakhir program akan menampilkan outputnya dalam satu baris.

2. Soal Unguided 2

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main
import "fmt"

func selectionSortAsc(arr []int, n int) {
    var idxMin, i, j int
```

```

        for i = 0; i < n-1; i++ {
            idxMin = i
            for j = i + 1; j < n; j++ {
                if arr[j] < arr[idxMin] {
                    idxMin = j
                }
            }
            if idxMin != i {
                arr[i], arr[idxMin] = arr[idxMin], arr[i]
            }
        }
    }

func selectionSortDesc(arr []int, n int) {
    var idxMax, i, j int
    for i = 0; i < n-1; i++ {
        idxMax = i
        for j = i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        if idxMax != i {
            arr[i], arr[idxMax] = arr[idxMax], arr[i]
        }
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        ganjil := make([]int, m)
        genap := make([]int, m)
        cntGanjil := 0
        cntGenap := 0

        for j := 0; j < m; j++ {
            var x int
            fmt.Scan(&x)
            if x%2 != 0 {
                ganjil[cntGanjil] = x
                cntGanjil++
            }
        }
    }
}

```

```

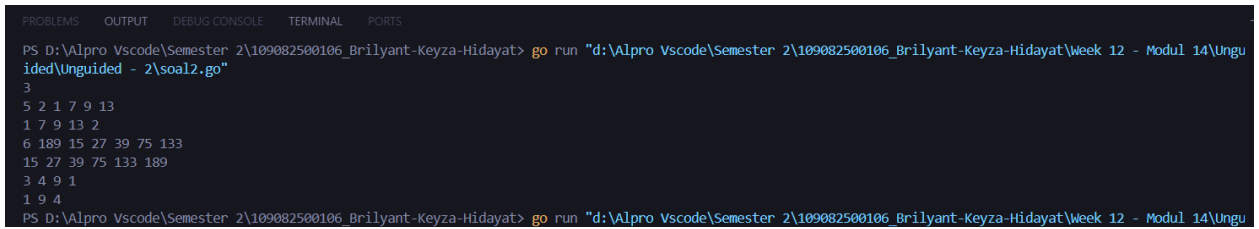
        } else {
            genap[cntGenap] = x
            cntGenap++
        }
    }

    selectionSortAsc(ganjil, cntGanjil)
    selectionSortDesc(genap, cntGenap)

    first := true
    for j := 0; j < cntGanjil; j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(ganjil[j])
        first = false
    }
    for j := 0; j < cntGenap; j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(genap[j])
        first = false
    }
    fmt.Println()
}
}

```

Output :



```

PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Unguided\Unguided - 2\soal2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Unguided\Unguided - 2\soal2.go"

```

Penjelasan :

Program diatas merupakan program yang di rancang untuk menyelesaikan masalah dengan memisahkan dan mengurutkan nomor rumah berdasarkan aturan ganjil dan genap. Cara kerja program ini adalah, pertama program meminta input jumlah daerah n yang akan diproses. Di dalam perulangan sebanyak n tersebut, program meminta input jumlah rumah m yang ada di daerah itu. Selanjutnya, program membaca satu per satu nomor rumah yang diinputkan.

Di dalam proses pembacaan ini, program langsung memisahkan data; jika nomor rumah merupakan bilangan ganjil, maka disimpan ke dalam array ganjil, namun jika genap, maka disimpan ke dalam array genap. Setelah semua nomor rumah selesai dikelompokkan, program memanggil dua fungsi pengurutan berbasis *Selection Sort*. Array ganjil diurutkan secara membesar (*ascending*), sedangkan array genap diurutkan secara mengecil (*descending*). Terakhir, program akan menampilkan output seluruh isi array ganjil yang sudah terurut membesar lebih dulu, lalu langsung dilanjutkan dengan menampilkan seluruh isi array genap yang sudah terurut mengecil dalam satu baris yang sama.

3. Soal Unguided 3

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 17 19 37 43	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main
import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
```

```

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func cekJarak(T *arrInt, n int) bool {
    var jarak, i int
    jarak = T[1] - T[0]
    for i = 2; i < n; i++ {
        if T[i]-T[i-1] != jarak {
            return false
        }
    }
    return true
}

func main() {
    var data arrInt
    var n, x int

    n = 0
    for {
        fmt.Scan(&x)
        if x < 0 {
            break
        }
        data[n] = x
        n++
    }

    insertionSort(&data, n)

    var i int
    for i = 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(data[i])
    }
    fmt.Println()

    if cekJarak(&data, n) {
        jarak := data[1] - data[0]
        fmt.Println("Data berjarak", jarak)
    }
}

```

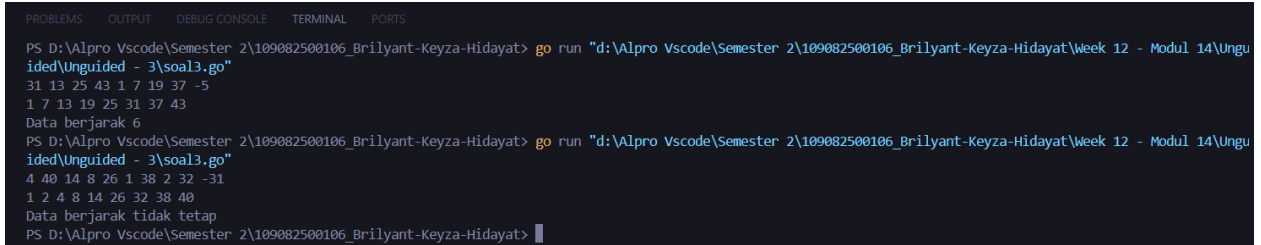


```

    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Ungu
ided\Unguided - 3\soal3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Ungu
ided\Unguided - 3\soal3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat>

```

Penjelasan :

Program ini dirancang untuk membaca sekumpulan data bilangan bulat positif, mengurutkannya, dan memeriksa apakah selisih antar angka yang berdekatan bernilai konstan atau sama. Cara kerja program ini adalah, pertama program meminta input bilangan bulat terus-menerus di dalam perulangan tanpa batas yang akan langsung berhenti jika pengguna menginputkan bilangan negatif. Selama input bernilai non-negatif, angka tersebut akan disimpan ke dalam array data dan menghitung jumlah total elemennya ke dalam variabel *n*. Setelah proses input selesai, program akan memanggil subprogram *insertionSort* untuk menyusun seluruh angka di dalam array secara membesar (*ascending*). Selanjutnya, program mencetak seluruh isi array yang sudah terurut rapi dalam satu baris. Terakhir, program akan memanggil fungsi *cekJarak* untuk memvalidasi selisih antar elemen, jika selisih setiap angka yang berdekatan selalu konsisten, maka program akan menampilkan status "Data berjarak" diikuti nilai jaraknya, namun jika selisihnya berubah-ubah, program akan menampilkan status "Data berjarak tidak tetap".

4. Soal Unguided 4

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
 terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
 Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
 eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
 dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
 rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```
package main
import "fmt"

const nMax int = 7919

type Buku struct {
    id        string
    judul     string
    penulis   string
    penerbit  string
    eksemplar int
    tahun     int
    rating    int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    var i int
    for i = 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id)
        fmt.Scan(&pustaka[i].judul)
        fmt.Scan(&pustaka[i].penulis)
        fmt.Scan(&pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar)
        fmt.Scan(&pustaka[i].tahun)
        fmt.Scan(&pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var idxMax, i int
    idxMax = 0
    for i = 1; i < n; i++ {
        if pustaka[i].rating > pustaka[idxMax].rating {
            idxMax = i
        }
    }
    fmt.Println(pustaka[idxMax].judul,
pustaka[idxMax].penulis, pustaka[idxMax].penerbit,
pustaka[idxMax].tahun)
}
```

```

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku
    var i, j int
    for i = 1; i <= n-1; i++ {
        temp = pustaka[i]
        j = i
        for j > 0 && temp.rating > pustaka[j-1].rating
        {
            pustaka[j] = pustaka[j-1]
            j = j - 1
        }
        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var i int
    batas := 5
    if n < 5 {
        batas = n
    }
    for i = 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    var mid, low, high int
    low = 0
    high = n - 1
    mid = -1

    for low <= high {
        mid = (low + high) / 2
        if pustaka[mid].rating == r {
            break
        } else if pustaka[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }

    if mid >= 0 && pustaka[mid].rating == r {
        fmt.Println(pustaka[mid].judul,
pustaka[mid].penulis, pustaka[mid].penerbit,

```

```

pustaka[mid].tahun, pustaka[mid].eksemplar,
pustaka[mid].rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating
seperti itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var nPustaka int
    var r int

    DaftarkanBuku(&pustaka, &nPustaka)

    fmt.Scan(&r)

    CetakTerfavorit(pustaka, nPustaka)

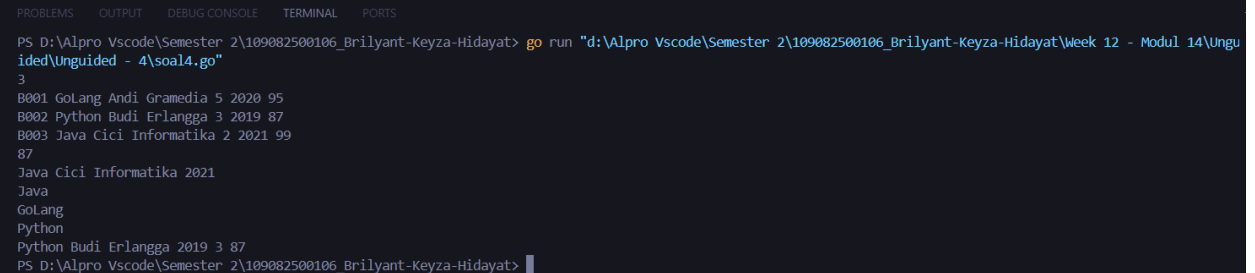
    UrutBuku(&pustaka, nPustaka)

    Cetak5Terbaru(pustaka, nPustaka)

    CariBuku(pustaka, nPustaka, r)
}

```

Output :



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat> go run "d:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat\Week 12 - Modul 14\Ungu
ided\Unguided - 4\soal4.go"
3
B001 GoLang Andi Gramedia 5 2020 95
B002 Python Budi Erlangga 3 2019 87
B003 Java Cici Informatika 2 2021 99
87
Java Cici Informatika 2021
Java
GoLang
Python
Python Budi Erlangga 2019 3 87
PS D:\Alpro Vscod\Semester 2\109082500106_Brilyant-Keyza-Hidayat>

```

Penjelasan :

Program ini dirancang untuk mengelola data koleksi buku di perpustakaan menggunakan representasi struktur data *Struct*. Cara kerja program ini adalah, pertama program memanggil prosedur `DaftarkanBuku` untuk meminta input jumlah buku `n` beserta seluruh detail atributnya, mulai dari ID, judul, penulis, penerbit, eksemplar, tahun, hingga rating buku tersebut. Selanjutnya, program meminta input satu bilangan bulat `r` yang merupakan target rating buku yang ingin dicari di akhir sesi. Setelah semua data buku tersimpan ke dalam array `pustaka`,

program mengesekusi fungsi `CetakTerfavorit` dengan memindai array untuk mencari dan menampilkan informasi buku yang memiliki rating tertinggi ke layar. Kemudian, program menjalankan subprogram `UrutBuku` untuk menyusun ulang seluruh daftar buku secara menurun (*descending*) berdasarkan ratingnya menggunakan metode *Insertion Sort*. Setelah data buku terurut rapi, program memanggil prosedur `Cetak5Terbaru` untuk menampilkan maksimal lima judul buku teratas yang memiliki perolehan rating tertinggi. Terakhir, program mengimplementasikan algoritma pencarian biner atau belah dua (*Binary Search*) melalui fungsi `CariBuku` untuk menemukan dan menampilkan data buku secara utuh yang nilainya sama dengan rating `r` yang dicari, atau menampilkan pesan khusus jika buku tidak ditemukan

E. Kesimpulan

Berdasarkan dari praktikum yang telah dipelajari, kesimpulannya materi pengurutan data atau *sorting* ini saya mempelajari dua algoritma , yaitu yang pertama Selection Sort yang bekerja dengan cara mencari nilai ekstrim (terkecil atau terbesar) pada sisa data lalu menukarnya ke posisi yang tepat , dan Insertion Sort yang bekerja dengan cara menyisipkan suatu nilai ke dalam rangkaian data yang sudah terurut menggunakan pencarian sekuensial. Kedua algoritma ini juga dapat disesuaikan untuk mengurutkan tipe data kompleks seperti *struct* dengan berfokus pada *field* atau atribut tertentu. Pemahaman mengenai konsep materi ini telah saya pelajari melalui implementasi beberapa program, yaitu penyusunan nomor rumah kerabat Hercules secara meningkat (Unguided 1), pemisahan sekaligus pengurutan nomor rumah ganjil dan genap (Unguided 2), pemeriksaan konsistensi selisih antar bilangan bulat positif (Unguided 3), serta pengelolaan data buku perpustakaan menggunakan metode Insertion Sort dan Binary Search (Unguided 4).